

OPENMOWER-TAF-ROS

OpenMower-TAF-ROS - Änderungsdocumentation - GPS Logging unter GPS State

Umsetzung der GPS-/RTK-Logging-Schnittstelle mit besonderem Fokus auf eine robuste spätere App-Integration

Ergebnis: Die GPS-Logging-Einstellungen, Steuerbefehle, Laufzeitdaten, Abschlussdaten und Validierungen werden fachlich unter `gps_state` bereitgestellt. `mower_logic` bleibt intern der zyklusabhängige Ausführer. Die App erhält damit eine eindeutige, dynamisch beschriebene und migrationsfähige Schnittstelle.

Angabe	Wert
Projektname	OpenMower-TAF-ROS
Dokumentenart	Änderungsdokumentation / API- und App-Integrationskonzept
Erstellungsdatum	2026-07-13
Verantwortlich	OpenMower-TAF-ROS Projektteam; technische Ausarbeitung: OpenAI
Version	v0.1
Status	Umgesetzt - statisch geprüft; ROS-/Catkin-Integrationsbuild ausstehend
Änderungsdatum	2026-07-13
Ablageort	OpenMower-TAF-ROS/_Dokumentation/
Referenzen	GPS_LOGGING_API.md; xbot_monitoring.cpp; mower_logic.cpp; Projektdokumentationsregeln vom 2026-06-18

Dokumentenstatus: Diese Fassung beschreibt den implementierten Stand im gelieferten Codepaket. Die vollständige Laufzeitprüfung muss im OpenMower-ROS-Umfeld erfolgen.

1. Ziel und Ergebnis der Änderung

Die bisher in mower_logic sichtbaren Satellite-Logging-Felder wurden fachlich in den GPS-State-Bereich verschoben. Dadurch muss die App nicht mehr zwischen GPS-Diagnose, GPS-Logging und internen Mäherlogik-Einstellungen wechseln. Die bestehende zyklusabhängige Ausführung bleibt erhalten, wird aber über eine neue öffentliche MQTT-Schicht unter gps_state angesprochen.

- Eine kanonische öffentliche Schnittstelle unter gps_state/logging/*.
- GPS-Logging-Standardwerte erscheinen in gps_state/settings/json in der Gruppe logging.
- Start, Stop und Abbruch sind Befehle und kein persistenter Bool-Schalter.
- Laufzeitstatus und letzte abgeschlossene Session werden getrennt und retained veröffentlicht.
- Einstellungsänderungen besitzen einen nachvollziehbaren forwarded/pending/applied-Ablauf.
- Interne satellite_logging_* Felder werden aus dem öffentlichen mower_logic-Settings-Payload entfernt, damit die App keine doppelten Bedienelemente erzeugt.
- Die bisherigen mower_logic-Topics bleiben während der Migration als Kompatibilitätsschicht erhalten.

App-Leitprinzip: Die App rendert Labels, Reihenfolge, Expertensicht, Enums, Topics und Payload-Schemata aus gps_state/settings/json. Der bestätigte retained Laufzeitstatus ist die alleinige Quelle für den Zustand der Schaltflächen.

1.1 Abgrenzung

Die Änderung verschiebt die öffentliche Zuständigkeit, nicht zwangsläufig den Prozessbesitz. mower_logic kennt weiterhin Docking-, Arbeits- und Flächenzustände und entscheidet daher intern, wann vorgemerkte Aufzeichnungen starten oder am Dock enden. xbot_monitoring bildet diese interne Funktion auf die neue GPS-State-API ab.

1.2 Zielarchitektur

Ebene	Verantwortung	Schnittstelle
App	Darstellung, Eingabe, Pending-Anzeige, Fehlerfeedback	nur gps_state/settings/* und gps_state/logging/*
xbot_monitoring	Öffentliche API, Validierung, Mapping, retained Status/Last	MQTT <-> ROS-Brücke
mower_logic	Triggerlogik, Prozessstart/-stop, Sessiondaten	/mower_logic/satellite_logging/*
record_satellites.sh	ROS-Topics aufzeichnen und Dateien sichern	RAM-Pfad -> persistenter Zielpfad

Architekturfluss

```

App
|
| MQTT gps_state/logging/*
v
xbot_monitoring -- Mapping/Validierung/Retain --> mower_logic
|
v
record_satellites.sh

```

2. Öffentlicher MQTT-Vertrag

Topic	Retained	Richtung	Verwendung in der App
gps_state/settings/json	ja	Mäher -> App	Dynamische Metadaten und bestätigte Einstellungswerte
gps_state/settings/set/session/json	nein	App -> Mäher	Standardwerte bis zum Neustart ändern
gps_state/settings/set/persistent/json	nein	App -> Mäher	Standardwerte dauerhaft speichern
gps_state/settings/validation/json	nein	Mäher -> App	Einstellungsänderung: forwarded/pending/applied/rejected
gps_state/logging/set/control/json	nein	App -> Mäher	Start, Stop oder Abbruch anfordern
gps_state/logging/validation/json	nein	Mäher -> App	Sofortige Syntax-/Zustandsvalidierung des Steuerbefehls
gps_state/logging/set/renew/json	nein	App -> Mäher	Settings, Status und letzte Session neu veröffentlichen
gps_state/logging/status/json	ja	Mäher -> App	Aktueller bestätigter Laufzeitstatus
gps_state/logging/last/json	ja	Mäher -> App	Letzte abgeschlossene oder abgebrochene Session

2.1 Warum Status und letzte Session getrennt sind

Der Live-Status kann nach einer Aufzeichnung wieder idle werden. Ohne ein separates retained last-Topic würde die App den zuletzt erzeugten Datensatz nach einem Neustart oder Reconnect nicht zuverlässig anzeigen können. last/json bleibt deshalb unabhängig vom aktuellen Startzustand erhalten.

2.2 Retain-Regeln für die App

- status/json und last/json sofort nach MQTT-Verbindung abonnieren.
- Bedienelemente erst nach dem ersten retained Status freigeben oder als noch nicht synchronisiert kennzeichnen.
- validation/json nicht als dauerhaften Zustand speichern; das Topic ist absichtlich nicht retained.
- Bei Reconnect logging/set/renew/json mit {} senden, falls Status oder Settings fehlen.

3. Dynamische GPS-Logging-Einstellungen

Öffentlicher Schlüssel	Typ / UI	Experte	Interne Zuordnung
logging_default_trigger	Enum-Auswahl	nein	satellite_logging_default_trigger
logging_default_mode	Enum-Auswahl	nein	satellite_logging_default_mode
logging_default_area_id	Text / Flächenauswahl	nein	satellite_logging_default_area_id
logging_output_path	Pfadtext	ja	satellite_logging_output_path
logging_ram_path	Pfadtext	ja	satellite_logging_ram_path
logging_script_path	Pfadtext	ja	satellite_logging_script_path
logging_container_name	Text	ja	satellite_logging_container_name

Wichtig: satellite_logging_enabled wird nicht als GPS-State-Einstellung übernommen. Ein dauerhafter Schalter wäre semantisch falsch, weil Start/Stop eine Laufzeitaktion ist. Die App sendet stattdessen control-Befehle und liest anschließend den bestätigten Status.

3.1 Zulässige Werte

Feld	Werte	Bedeutung
logging_default_trigger	ad_hoc	Sofort starten
	next_cycle	Beim nächsten passenden Arbeits-/Dockingereignis starten
	area_id	Bei aktiver Arbeit in einer bestimmten Fläche starten
logging_default_mode	from_start_to_docking	Bei Arbeitsstart beginnen, beim Andocken beenden
	from_docking_to_docking	Beim Verlassen des Docks beginnen, beim nächsten Andocken beenden
	until_docking	Nach Start bis zum Andocken aufzeichnen
	manual	Nur per Stop/Cancel beenden
	area_only	Interner Spezialmodus; nicht automatisch am Dock beenden
	area_to_docking	Interner Spezialmodus; Fläche bis Andocken

4. Steuerbefehle und App-Interaktion

Alle Befehle werden als JSON an `gps_state/logging/set/control/json` gesendet. `request_id` ist optional und wird in der unmittelbaren Validierungsantwort gespiegelt. Dadurch kann die App mehrere Aktionen eindeutig korrelieren, ohne interne Session-IDs vorwegzunehmen.

Sofortstart

```
{"command": "start", "trigger": "ad_hoc", "mode": "until_docking", "request_id": "gps-ui-42"}
```

Nächsten Mähzyklus vormerken

```
{"command": "start", "trigger": "next_cycle", "mode": "from_start_to_docking"}
```

Flächenbezogener Start

```
{"command": "start", "trigger": "area_id", "mode": "until_docking", "area_id": "3"}
```

Laufende Aufzeichnung regulär beenden

```
{"command": "stop"}
```

Vormerkung abbrechen oder laufende Session als cancelled beenden

```
{"command": "cancel"}
```

Befehl	Zulässig	Bestätigungsquelle	App-Verhalten
start	kein aktiver Request	validation + status	Button sperren, pending anzeigen, auf armed/running/error warten
stop	vorzugsweise running	status	auf finished/error warten; last-Session aktualisieren
cancel	armed oder running	status + ggf. last	armed abbrechen; laufende Aufnahme stoppen und als cancelled erhalten

Dateierhalt bei cancel: Wurde die Aufnahme bereits gestartet, beendet cancel den Loggerprozess, die vom Skript erzeugten Dateien bleiben erhalten und `last/json` wird mit `result=cancelled` aktualisiert. Cancel ist daher kein Löschen von Daten.

4.1 Unmittelbare Steuerbefehlsvalidierung

```
{
  "schema": "openmower.gps_state.logging.validation.v1",
  "type": "control",
  "valid": true,
  "status": "forwarded",
  "accepted": [{"command": "start"}],
  "rejected": [],
  "request_id": "gps-ui-42",
  "published_at": 1783951200.0
}
```

forwarded bedeutet nur, dass die Brücke den Befehl akzeptiert und an `mower_logic` weitergegeben hat. Die App darf daraus noch keinen laufenden Logger ableiten. Erst `status/json` bestätigt `armed`, `running`, `finished` oder `error`.

5. Laufzeitzustände und Schaltflächenlogik

Bedingung	runtime.state	request_active	armed	running	Empfohlene Primäraktion
Keine Anforderung	idle / finished	false	false	false	Start
Vorgemerkt	armed	true	true	false	Abbrechen
Aufzeichnung läuft	running	true	false	true	Stop

Bedingung	runtime.state	request_active	armed	running	Empfohlene Primäraktion
Erfolgreich beendet	finished	false	false	false	Start / Details
Abgebrochen	idle oder finished	false	false	false	Start / letzte Session
Fehler	error	false oder true	false	false	Fehler anzeigen / Renew

Zustandsfolge

```
idle -> start -> armed -> Trigger erfüllt -> running -> Dock/stop -> finished
idle -> start(ad_hoc) -----> running -> cancel -> last.result=cancelled
armed -> cancel -> idle
```

- Die UI folgt runtime.request_active, runtime.armed und runtime.running; nicht dem zuletzt gesendeten Befehl.
- Ein zweiter Start wird abgewiesen, solange eine Anforderung aktiv, vorgemerkt oder laufend ist.
- duration_s wird mit jeder Statusnachricht aktualisiert. Für einen flüssigen Timer berechnet die App die laufende Dauer lokal aus started_at.
- Bei error wird error im sichtbaren Fehlerbereich angezeigt; technische Pfade und PID bleiben in der Expertensicht.

6. Laufzeitstatus: gps_state/logging/status/json

```
{
  "schema": "openmower.gps_state.logging.v1",
  "type": "status",
  "published_at": 1783951200.0,
  "status": "running",
  "severity": 1,
  "summary": "GPS-Aufzeichnung läuft",
  "runtime": {
    "state": "running", "request_active": true,
    "request_origin": "command", "armed": false, "running": true,
    "pid": 1842, "session_id": "20260713_143522",
    "requested_at": "2026-07-13T14:35:21Z",
    "started_at": "2026-07-13T14:35:22Z",
    "finished_at": null, "duration_s": 142.5, "stop_reason": null
  },
  "request": {"trigger": "ad_hoc", "mode": "until_docking", "target_area_id": null},
  "storage": {
    "ram_path": "/dev/shm/openmower_satellite_logs",
    "output_path": "/home/openmower/recordings/logs",
    "files": ["gps_position_20260713_143522.log"]
  },
  "implementation": {
    "script_path": "/home/openmower/scripts/record_satellites.sh",
    "container_name": null, "legacy_setting_enabled": false
  },
  "error": null
}
```

Pfad	Bedeutung	App-Nutzung
status / runtime.state	Aktueller Controllerzustand	Status-Badge und Zustandsmaschine
severity	0 normal, 1 aktiv/info, 4 Fehler	Icon/Farbe; nie als einzige Information
summary	Menschenlesbare Kurzbeschreibung	Überschrift der Statuskarte
runtime.request_active	Offene Laufzeitanforderung	Start sperren
runtime.request_origin	command oder legacy_setting	Migration/Diagnose, meist Expertensicht
runtime.armed / running	Vorgemerkt bzw. Prozess aktiv	Cancel- oder Stop-Schaltfläche
runtime.session_id	Eindeutige Logging-Session	Details und Dateizuordnung
requested_at / started_at / finished_at	UTC-ISO-8601-Zeitpunkte	Zeitdarstellung in lokaler Zeitzone
runtime.duration_s	Dauer beim Publikationszeitpunkt	Anzeige; laufend lokal fortschreiben
runtime.stop_reason	docked, manual_stop, cancelled, error, legacy_disabled	Abschlussbeschreibung
request.*	Trigger, Modus, Ziel-Fläche	Karte Aktuelle Anforderung
storage.*	RAM-/Zielpfad und erwartete Dateien	Dateiliste; Pfade nur Expertensicht
implementation.*	Skript, Container, Legacy-Flag	Nur Diagnose/Expertensicht
error	Fehlertext oder null	Sichtbare Fehlermeldung

7. Letzte Session: `gps_state/logging/last/json`

`last/json` wird nur aus einem Status mit Session-ID und Abschlusszeit erzeugt. Es bleibt retained verfügbar, auch wenn `status/json` wieder `idle` meldet.

```
{
  "schema": "openmower.gps_state.logging.last.v1",
  "type": "last",
  "result": "finished",
  "session_id": "20260713_143522",
  "requested_at": "2026-07-13T14:35:21Z",
  "started_at": "2026-07-13T14:35:22Z",
  "finished_at": "2026-07-13T15:17:04Z",
  "duration_s": 2502.0,
  "stop_reason": "docked",
  "trigger": "next_cycle",
  "mode": "from_start_to_docking",
  "target_area_id": null,
  "output_path": "/home/openmower/recordings/logs",
  "files": ["gps_position_20260713_143522.log"],
  "error": null
}
```

- `result` besitzt `finished`, `cancelled` oder `error`.
- Die App zeigt `last` unabhängig vom Live-Status an.
- Dateien sind aktuell erwartete Session-Dateinamen; die App sollte fehlende Dateizugriffsfunktionen tolerant behandeln.
- Unbekannte zusätzliche Felder müssen ignoriert werden, damit das Schema erweiterbar bleibt.

8. Einstellungsänderungen: Pending- und Bestätigungsablauf

Die öffentlichen logging_* Schlüssel werden von xbot_monitoring auf bestehende mower_logic-Felder abgebildet. Weil die interne Settings-Verarbeitung asynchron ist, erfolgt eine zweistufige Rückmeldung.

Schritt	Nachricht / Zustand	App-Reaktion
1	App sendet logging_* Wert an set/session oder set/persistent	Control lokal als pending markieren
2	settings/validation: status=forwarded, pending=true	Wert noch nicht als bestätigt speichern
3	Brücke leitet gemappten satellite_logging_* Wert weiter	Keine zusätzliche UI-Aktion
4	mower_logic publiziert bestätigten Settings-Stand	Brücke gleicht Soll/Ist ab
5	gps_state/settings/json enthält bestätigte public Werte	UI-Control mit Serverwert aktualisieren
6	settings/validation: status=applied, pending=false	Pending beenden und Erfolg anzeigen
Fehler	status=rejected oder Timeout	Alten bestätigten Wert behalten; Fehlermeldung / Renew anbieten

Beispiel für gps_state/settings/set/persistent/json

```
{
  "logging_default_trigger": {"value": "area_id"},
  "logging_default_area_id": {"value": "3"}
}
```

App-Timeout: Wird applied nicht innerhalb eines app-seitig definierten Zeitfensters empfangen, darf die App nicht stillschweigend Erfolg annehmen. Sie zeigt den Zustand als ungeklärt an und bietet logging/set/renew/json beziehungsweise gps_state/settings/set/renew/json an.

9. Verbindlicher Leitfaden für die spätere App-Umsetzung

Bereich	Umsetzungsvorgabe
Dynamisches Rendering	Gruppe, label, description, order, type, enum, expert und topic aus gps_state/settings/json lesen.
Quelle der Wahrheit	Buttons und Status ausschließlich aus retained status/json ableiten.
Startaktion	Nach Publish sofort pending anzeigen; erst armed/running/error beendet pending.
Stop/Cancel	Aktion bis zum nächsten bestätigten Status sperren; wiederholtes Senden verhindern.
Korrelation	Für Steuerbefehle request_id verwenden und validation darauf zuordnen.
Live-Dauer	Aus started_at lokal fortschreiben; bei neuer Statusnachricht synchronisieren.
Zeitangaben	UTC-ISO-8601 parsen und in lokaler Zeitzone anzeigen; Rohwert in Details erhalten.
Expertensicht	RAM-/Ziel-/Skriptpfad, Container, PID und Legacy-Flag nicht in der Standardansicht zeigen.
Fehler	summary + error sichtbar anzeigen; severity nicht nur über Farbe kommunizieren.
Schema-Toleranz	Unbekannte Felder ignorieren; schema-Wert protokollieren; erforderliche Felder defensiv lesen.
Reconnect	Settings, Status und Last abonnieren; bei fehlenden retained Daten Renew senden.
Migration	Keine neuen Zugriffe auf settings/mower_logic/satellite_logging/* oder satellite_logging_* UI-Felder.

10. Empfohlener Bildschirmaufbau

UI-Block	Standardansicht	Expertenerweiterung
Kopfkarte	summary, Zustandsbadge, lokale Laufzeit, Start/Stop/Cancel	request_origin, raw state
Aktuelle Anforderung	Trigger, Modus, Ziel-Fläche, requested_at	keine
Aktuelle Session	Session-ID, Start, Dateianzahl	PID, RAM-Pfad, Skript, Container
Letzte Session	result, Dauer, stop_reason, Zeiten, Dateien	Zielpfad und raw JSON
Standardwerte	Trigger, Modus, Flächen-ID	RAM-/Ziel-/Skript-/Containerpfade
Rückmeldung	Pending, Erfolg, Fehler, Renew	Validierungsdetails

Barrierearme Statusdarstellung: armed, running, finished und error werden durch Text und Symbol dargestellt. Farbe ist nur ergänzend. Fehlermeldungen bleiben kopierbar und die primäre Aktion besitzt eine eindeutige Beschriftung.

10.1 Beispielhafte Button-Entscheidung

```
if status.runtime.running:
    primary = "Stop"
elif status.runtime.armed or status.runtime.request_active:
    primary = "Abbrechen"
else:
    primary = "Start"

# Nach einem Publish nicht optimistisch umschalten:
# pending bleibt aktiv, bis status/json den neuen Zustand bestätigt.
```

10.2 Empfohlene Komponenten

- GpsLoggingStatusCard: Status, Laufzeit, Primäraktion und Fehler.
- GpsLoggingRequestCard: Trigger, Modus, Fläche und Anforderungszeit.
- GpsLoggingLastSessionCard: Ergebnis, Dauer, Beendigungsgrund und Dateien.
- GpsLoggingDefaultsForm: dynamisch aus der settings-Metadatenstruktur.
- GpsLoggingExpertPanel: technische Pfade, PID, Container und Rohdaten.
- MqttCommandPendingState: request_id, Timeout, Validation und Statusbestätigung.

11. Migration und Rückwärtskompatibilität

Alt	Neu	Migrationsverhalten
settings/mower_logic/satellite_logging/json	gps_state/logging/status/json	Alt bleibt veröffentlicht; neue App nutzt nur Neu.
settings/mower_logic/satellite_logging/set/control/json	gps_state/logging/set/control/json	Alt bleibt abonnierbar; neue App nutzt nur Neu.
settings/mower_logic/satellite_logging/set/renew/json	gps_state/logging/set/renew/json	Alt bleibt abonnierbar; Neu aktualisiert zusätzlich Settings/Last.
satellite_logging_* in settings/mower_logic/json	logging_* in gps_state/settings/json	Interne Felder werden im öffentlichen mower_logic-Payload ausgefiltert.
satellite_logging_enabled	control-Befehl + runtime status	Nicht als neuer Toggle übernehmen.

Die Kompatibilitätsschicht erlaubt eine schrittweise App-Umstellung. Nach erfolgreicher Migration können die alten Topics in einer späteren Hauptversion mit eigener Deprecation-Phase entfernt werden. Das ist nicht Bestandteil von v0.1.

12. Technische Umsetzung im Code

Datei	Änderung
src/lib/xbot_monitoring/src/xbot_monitoring.cpp	Neue Topics, Settings-Metadaten, Key-Mapping, Validierung, Pending-Abgleich, Status-/Last-Transformation, Cache und Legacy-Filterung.
src/lib/xbot_monitoring/GPS_LOGGING_API.md	Kanonischer API-Vertrag und konkrete App-Integrationsregeln.
src/lib/xbot_monitoring/README.md	Verweis auf API und zentrale Migrationshinweise.
src/mower_logic/src/mower_logic/mower_logic.cpp	Laufzeitanforderung von Legacy-Enable entkoppelt; request_origin, requested_at, stop_reason und Duplicate-Start-Schutz ergänzt.
src/mower_logic/cfg/MowerLogic.cfg	Kennzeichnung der Felder als interne Runtime-Konfiguration; öffentliche API liegt unter gps_state.
src/mower_logic/scripts/record_satellites.sh	Hinweis auf öffentliches gps_state-Mapping ergänzt.
SATELLITE_LOGGING_MOWER_LOGIC_README.txt	Neue öffentliche Topics, Statusfelder und App-Regeln dokumentiert.
SATELLITE_LOGGER_MQTT_README.txt	Kompakte MQTT-Migrationsreferenz aktualisiert.

13. Prüfung, bekannte Grenzen und Abnahme

Prüfung	Ergebnis	Bemerkung
git diff --check	bestanden	Keine Whitespace-Fehler in den geänderten Textdateien.
Shell-Syntax record_satellites.sh	bestanden	bash -n ohne Fehler.
C++ Klammer-/String-Balance	bestanden	Statische lexikalische Prüfung beider geänderten C++-Dateien.
API-Contract-Strings	bestanden	Alle neuen Topics, Schemas und zentralen Settings-Schlüssel vorhanden.
DOCX-Renderprüfung	bestanden nach Auslieferungs-QA	Alle Seiten als PNG gerendert und visuell geprüft.
PDF-Renderprüfung	bestanden nach Auslieferungs-QA	PDF gerendert und visuell geprüft.
ROS-/Catkin-Build	nicht ausgeführt	In der Bearbeitungsumgebung ist keine /opt/ros-Installation vorhanden.
Hardware-/MQTT-Laufzeittest	ausstehend	Auf Zielsystem mit ROS, MQTT, mower_logic und Logger-Skript durchführen.

Abnahmebedingung: Vor Produktivfreigabe müssen Start, Armed-Trigger, Docking-Stop, manueller Stop, Cancel, Reconnect, persistente Settings und Fehlerfälle auf einem OpenMower-Zielsystem getestet werden.

13.1 Empfohlene Laufzeit-Testfälle

- Sofortstart ad_hoc: validation=forwarded, anschließend status.running=true, danach Stop und last.result=finished.
- next_cycle: status.armed=true, Start bei aktivem Arbeitszustand, Stop beim Andocken mit stop_reason=docked.
- area_id: nur bei passender Fläche starten; fehlende oder ungültige Fläche ablehnen beziehungsweise nicht starten.
- Doppelstart: zweiter Start wird abgewiesen, Session-ID und Dateiliste bleiben unverändert.
- Cancel im armed-Zustand: Rückkehr zu idle ohne neue last-Session.
- Cancel während running: Logger stoppt; last.result=cancelled und erzeugte Dateien bleiben gelistet.
- Persistente Änderung: forwarded/pending -> bestätigte settings -> applied/pending=false.
- MQTT-Reconnect: retained status und last erscheinen ohne neue Aufnahme.
- Fehlerpfade: fehlendes Skript, nicht beschreibbarer Zielpfad und Prozessabbruch sichtbar in error.

14. Rollback und Weiterentwicklung

Ein Rollback ist möglich, indem die acht geänderten Quell-/Dokumentationsdateien auf den Ausgangsstand zurückgesetzt werden. Da keine Datenmigration des persistenten Settings-Speichers erzwungen wird und die internen satellite_logging_* Schlüssel bestehen bleiben, bleiben vorhandene Werte erhalten.

- Kurzfristig: gps_state/logging/* deaktivieren und Legacy-Topics weiterverwenden.

- Langfristig: optional einen eigenen gps_logging-ROS-Node einführen; der öffentliche MQTT-Vertrag kann dabei unverändert bleiben.
- Künftige Schemaerweiterungen unter gleicher Hauptversion nur additiv durchführen; bestehende Felder nicht umdeuten.
- Bei inkompatiblen Änderungen neues Schema openmower.gps_state.logging.v2 und dokumentierte Übergangsphase verwenden.

15. Änderungshistorie

Version	Datum	Bearbeiter	Status	Änderung
v0.1	2026-07-13	OpenAI / OpenMower-TAF-ROS Projektteam	Umgesetzt - in Prüfung	Erstimplementierung der gps_state-Logging-API, App-Leitfaden und Auslieferungsdokumentation.

Gültiger Stand: Die in diesem Dokument beschriebenen Topics und Felder entsprechen dem Codepaket v0.1 vom 2026-07-13. Für die App ist `src/lib/xbot_monitoring/GPS_LOGGING_API.md` die kompakte technische Referenz; dieses Dokument ergänzt Architektur, UX und Abnahme.